上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/10

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

软件运行时

模块 5 · 重新认识软件（2/4）

饮 水 思 源 • 爱 国 荣 校

SJTU

重新认识软件 – 软件运行时

本讲位置：从「实例」到「进程」

上一讲 · L-05

软件的生命周期：制品轴与实例轴

实例会被频繁替换，因此须优雅停止、快速启动、不保存状态

遗留问题：启动时间为何是硬约束

本讲问题

实例轴上处于「服务中」的实例，在机器上以什么形式存在？占用哪些资源？上限由哪种资源决定？

引例

L-04 遗留的问题：选课高峰时，最先达到上限的是 Tomcat 线程池还是数据库行锁？

第三节解答，课程实践以数据验证

本讲结构

1 服务在机器上的形态

2 语言运行时

3 并发模型与吞吐

插曲：延迟的正确表述 · 思考题 2 发布

4 运行时的层级

5 课程实践：单机 Tomcat 的运行时画像

本模块 重新认识软件（L-05 ~ L-08）：制品（L-05）· 进程（本讲）· 资源（L-07）· 指标（L-08）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、服务在机器上的形态

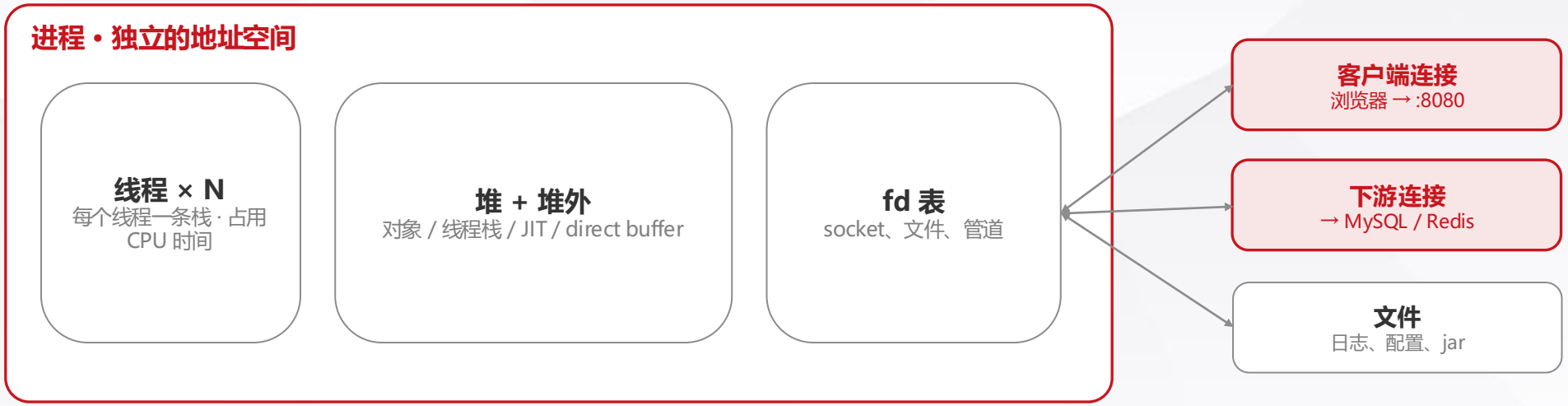
第六讲 · 软件运行时

饮水思源 • 爱国荣校

SJTU

操作系统视角 • 进程

从「实例」到「进程」



线程
占用 CPU 时间与栈内存

内存
堆与堆外占用物理页；依赖页缓存

fd
对应连接与打开的文件

要点 本节依次讨论这三类资源，均可在 /proc/<pid>/ 下观察；各自的上限在 L-07 讨论

来源：proc(5) man page；B. Gregg, Systems Performance, 2nd ed. (2020), Ch. 3

CPU：线程与调度

线程是调度单位

内核按时间片轮流将线程调度到核上

可运行线程数超过核数时即产生排队

利用率（核处于忙碌状态）≠ 饱和（有线程在等待核）

上下文切换

直接开销为微秒级，间接开销是 CPU 缓存失效

线程数超过一定程度后，切换本身开始显著消耗 CPU

如何解读指标

一个进程的 %CPU 可以超过 100%（4 核最多 400%）

user 为应用代码的执行时间，sys 为内核代为执行的时间

load average 统计处于可运行与不可中断状态的任务数

问题	原因	对策
CPU 利用率 60%，但响应持续变慢	观察的是平均值；部分核或部分时段已饱和，或线程在等待锁 / I/O	检查运行队列与 load average、线程状态以及 sys / user 比例
线程数增至 2000，吞吐量反而下降	上下文切换与缓存失效消耗了 CPU	线程数应依据核数与阻塞比例确定，而非并发用户数（第三节）

要点 利用率表示核处于忙碌状态，饱和表示有线程在等待核；响应变慢时，应先区分是在等待 CPU 还是其他资源

来源：B. Gregg, Systems Performance, Ch. 6 CPUs；Linux sched(7)

内存：进程占用的组成

进程的 RSS（实际占用的物理内存）

堆	堆外	共享库
运行时管理的对象	线程栈 · 元数据 · JIT 代码 · direct buffer	libc、JVM 本体

虚拟内存是已申请的地址空间，RSS 才是实际占用的物理内存

页缓存
内核利用空闲内存缓存文件，可回收

不计入进程，但服务性能依赖于它

问题	原因	对策
free 显示空闲内存接近耗尽，服务却运行正常	空闲内存被内核用作页缓存，需要时会释放	以 available 为准，而非 free
堆设为 2 GB，进程却占用 3 GB，在容器中被终止	堆只是一部分：线程栈、元数据、JIT 代码、direct buffer 均位于堆外	容器限制 = 堆 + 堆外 + 余量；JDK 10+ 读取 cgroup 限制以设定默认堆大小
进程突然消失，退出码 137	超过 cgroup 内存限制，被 OOM killer 终止（137 = 128 + 9，即 SIGKILL）	检查 dmesg / OOMKilled；核对限制与实际占用

要点 服务所需内存不等于 -Xmx，而是 RSS 的上限加上页缓存的需求

来源：B. Gregg, Systems Performance, Ch. 7 Memory；Linux proc(5)（/proc/meminfo、/proc/<pid>/status）

操作系统视角 • fd

连接与文件描述符

连接数 ≠ 请求数

每个 TCP 连接、每个打开的文件都占用一个 fd；进程的 fd 数有上限（ulimit -n，常见默认值 1024）

keep-alive 空闲连接即使不发送请求也占用 fd 与缓冲区：2000 人 × 每人数条 = 上万个 fd

连接的两端

客户端连接（浏览器 → Tomcat）与下游连接（Tomcat → MySQL / Redis）各自占用 fd

短连接频繁建立与关闭会产生大量 TIME_WAIT，并可能耗尽临时端口

问题	原因	对策
高峰时报 Too many open files，CPU 却较空闲	fd 上限 1024 被空闲 keep-alive 连接占满	提高进程与系统的 fd 上限；缩短 keep-alive 超时；在前端部署反向代理以收敛连接
新连接超时，已有连接正常	服务端 maxConnections 达到上限，新连接在 accept 队列中排队	检查 ss -s / ss -ltn 中的 backlog；调整 maxConnections 与 backlog；仍不足时横向扩展

回顾 思考题 1 中 Nginx 的一项隐含收益：由其承接上万条客户端连接，后端只需处理少量复用的上游连接

来源：Linux ss(8)、ulimit；Apache Tomcat 10.1 HTTP Connector 文档（maxConnections、acceptCount）

观测方法速查

观测目标	命令	关注指标
进程占用的 CPU / 内存	top；ps -o pid,nlwp,rss,%cpu	%CPU（可超 100%）、RSS、线程数 nlwp
CPU 是忙碌还是存在排队	uptime；vmstat 1	load average；r 列（可运行）、b 列（阻塞）
内存实际剩余量	free -m	available，而非 free
fd 数与连接数	ls /proc/<pid>/fd | wc -l；ss -tanp	fd 数；ESTAB / TIME-WAIT 计数
线程正在执行的操作	top -H；jstack；pprof	哪些线程占用 CPU；多少线程在等待锁或 I/O

要点 这些即 2025 实验 2 使用的工具；L-07 讨论各指标的上限，实验中自行测量

来源：各命令 man page；2025 实验 2 指导书

操作系统视角 · 演示

现场演示：观察一个真实进程（3 分钟）

演示内容

观察对象：一个空载的 Tomcat（或 2025 实验 3 的 Go 后端）

① **top -H -p <pid>**

线程数、每个线程的 %CPU、状态 S / R

② **grep -E "VmRSS|Threads" /proc/<pid>/status**

RSS 与线程数，并与 -Xmx 对照

③ **free -m**

free 与 available 的差值

④ **ls /proc/<pid>/fd | wc -l；ss -s**

空载时已有数十个 fd；连接的状态分布

随后施加 10 秒负载并再次观察

wrk -t4 -c200 -d10s http://<host>:8080/

预期结果

线程数不变（线程池已预分配），但 R 状态的线程增多

RSS 略升，远小于 200 × 1 MB：线程栈按需提交

ESTAB 连接约 200，压测结束后出现大量 TIME-WAIT

记录这四个数值，课程实践中将用于计算上限

说明 只关注数值，不展开工具用法；每个数值对应第 5 ~ 8 页中的一类资源

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、语言运行时

第六讲 · 软件运行时

饮水思源 • 爱国荣校

SJTU

JVM：堆、GC、JIT、线程

堆与 GC

对象分配在堆中，按代回收

GC 需要停顿（STW），停顿时长取决于回收器与堆大小

「堆越大越好」不成立

JIT

先解释执行，后编译热点

启动较慢，预热后峰值高

L-05 中「启动时间是硬约束」的根源之一

平台线程

1:1 对应 OS 线程，各有一条栈（预留 1 MB，按需提交）

200 个开销可接受，2 万个则开销显著

虚拟线程（JDK 21）

M:N 映射到少量 OS 线程，阻塞时让出载体线程

可创建百万级：以阻塞式写法获得接近非阻塞的开销

第三节说明它改变了什么、未改变什么

问题	原因	对策
容器中的 JVM 被意外终止	-Xmx 之外还有元空间、代码缓存、线程栈、direct buffer	容器限制 ≥ 堆 + 堆外 + 余量；使用 JDK 10+ 的容器感知
新发布的实例在最初几分钟明显偏慢	JIT 尚未完成预热	在就绪检查通过前完成预热（L-05）；或以 GraalVM 原生镜像替代 JIT

要点 JVM 的资源画像：内存基线高、启动需预热、线程映射到 OS 线程；每一项都有代价，也都有相应的新方案

来源：JEP 444 Virtual Threads；JDK 容器支持（JDK-8146115）；HotSpot GC Tuning Guide

Go · Node · Python：各自的并发单位

Go

goroutine：M:N 调度到 OS 线程，初始栈为 KB 级，十万级并发较为常见

并发 GC，停顿短；静态链接二进制，启动为毫秒级；自动利用多核

常见问题：goroutine 泄漏；GC 仍消耗 CPU

2025 实验 2、3 的后端即采用 Go

Node.js

单线程事件循环处理网络 I/O，libuv 线程池处理文件 I/O 等；并发依赖非阻塞

一段 CPU 密集代码会阻塞所有请求 → 使用 worker threads 或多进程

利用多核需启动多个进程

Python

GIL：同一时刻只有一个线程执行字节码（3.13 起提供可选的无 GIL 构建）

I/O 并发使用 asyncio，CPU 并行依赖多进程

典型部署 = 多个 worker 进程 × 少量线程

要点 并发单位不同，单实例可承载并发量的计算方法也不同；下一页汇总对比

来源：Go runtime scheduler 设计文档；Node.js docs, The Node.js Event Loop；PEP 703

同一服务的不同资源画像

运行时	并发单位	内存基线	启动时间	多核	峰值吞吐
JVM（JIT）	平台线程 / 虚拟线程	高（百 MB 起）	秒级 + 预热	自动	高
GraalVM 原生镜像	同上	低	毫秒级	自动	略低（无 JIT）
Go	goroutine	低（十 MB 级）	毫秒级	自动	高
Node.js	事件循环	中	亚秒	多进程	I/O 高，CPU 低
Python	进程 × 线程	中	亚秒	多进程	低

选择运行时即选择资源画像：同一选课服务，以 JVM 实现时一台机器可部署 2 个实例，以 Go 实现可部署 10 个；这正是 L-07 与模块 6 要核算的内容

回顾 L-03：Quarkus / GraalVM 改善启动时间与内存，代价是峰值吞吐与构建时间；表中数值为量级示意

判断 这项技术解决了什么问题，又增加了什么成本？

来源：L-03 讲义（Quarkus 对比）；GraalVM Native Image 文档

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

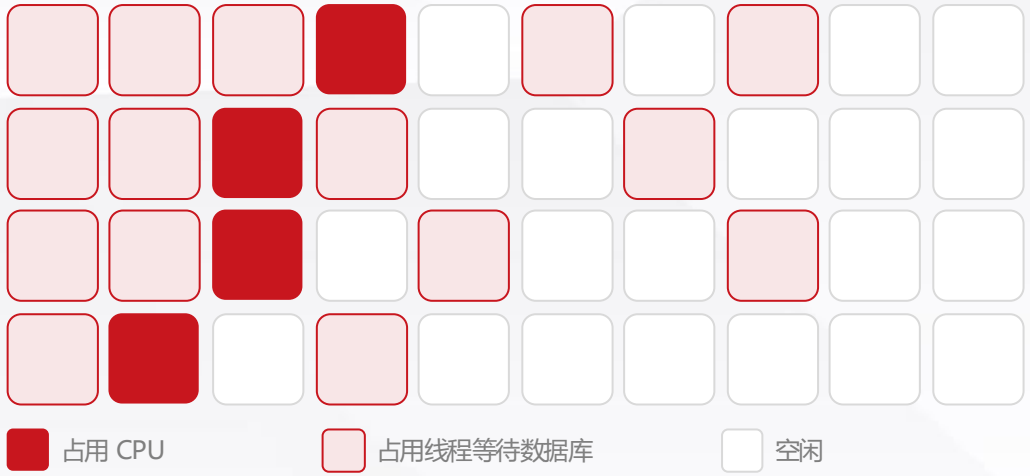
三、并发模型与吞吐

第六讲 · 软件运行时

饮 水 思 源 • 爱 国 荣 校

SJTU

一请求一线程



maxThreads = 200：每个请求占用一个线程，等待数据库返回期间该线程不消耗 CPU，但也不释放

Little’ s Law

在途请求 L = λ × W（W 为平均耗时）

一请求一线程 → 并发度上限 = 线程数 → λ_max = 线程数 / W

估算

200 线程、W = 50 ms → 4000 rps

数据库变慢使 W = 500 ms → 仅 400 rps

线程池先耗尽，随后排队，最终超时

要点 线程池的吞吐上限不是常数，它随请求的等待时间变化：等待越久，有效容量越小

来源：J. D. C. Little (1961)；Apache Tomcat 10.1 HTTP Connector（maxThreads 默认 200）

排队普遍存在

accept 队列
内核 backlog / acceptCount 100

→

线程池
maxThreads 200，满后排队

→

连接池
等待空闲的数据库连接

→

锁等待
热点行，同一时刻仅一个事务持有

每一级既是缓冲，也增加延迟；有界队列 + 超时 = 背压；无界队列会导致延迟与内存无限增长

问题	原因	对策
高峰过去很久，服务仍然缓慢	队列中积压的请求仍在处理，而客户端早已放弃	使用有界队列；请求携带截止时间，过期即丢弃
加大线程池后超时反而增多	排队位置从线程池转移到数据库	线程数依据下游容量确定；真正的瓶颈在下游

回顾思考题 1 第二幕：单机时以 p50 2.1 s 估算，在途请求约 5000 个（偏低估计），几乎全部在排队；重载下响应时间主要由排队构成

要点 队列是必要的缓冲，也是延迟的来源；无界队列最终会耗尽机器资源

来源：B. Gregg, Systems Performance, Ch. 2（排队论）；Tomcat acceptCount 默认 100

非阻塞与事件循环

epoll / NIO

少量线程管理大量连接；等待 I/O 时不占用线程

并发上限由线程数转变为内存与 fd

但 CPU 密集仍受核数限制

事件循环中执行 50 ms 的计算，所有连接都需等待 50 ms

Node.js 的典型故障

Tomcat 与虚拟线程

Tomcat NIO 连接器以此方式接收连接（acceptor + poller），业务处理仍为一请求一线程

虚拟线程使阻塞式写法获得接近非阻塞的开销，基本消除了线程数这一层的上限

要点 非阻塞改变的是线程这一层的上限；CPU、连接池与锁的上限均未改变（课程实践第 4 题）

来源：Linux epoll(7)；Tomcat NIO connector 文档；JEP 444

连接池：更稀缺的资源

数据库端同样以线程处理连接

MySQL 一连接一线程，max_connections 默认 151

因此连接池并非越大越好：HikariCP 建议约为数据库核数 × 2，默认值 10

连接池吞吐上限 = 池大小 / 平均占用时长

10 个连接、每次 8 ms → 1250 QPS，与应用线程数无关

连接池耗尽首先表现为延迟（等待连接），而非错误

问题	原因	对策
连接池由 10 调至 100，吞吐量不变，数据库 CPU 急剧上升	数据库端的并行度有限，多出的连接在数据库内部排队	连接池大小依据数据库处理能力确定，而非应用线程数
偶发慢请求，但数据库本身不慢	请求在等待获取连接	检查连接池的等待指标；缩短每次占用时长（缩短事务）或适度扩大连接池

要点 监控中应包含「等待获取连接」的时间；连接池常常是最先达到的上限

来源：HikariCP wiki, About Pool Sizing；MySQL 8.4 Reference, max_connections

解答 L-04 的问题：线程池还是行锁?

热点行行锁
提交被串行化

→ **300 TPS**
单机写吞吐上限

→ **线程池 200**
提交在此排队

→ **accept 队列 100**
第二层排队

→ **超时 · 重试**
其余请求滞留在客户端

← 根因在此端

表象在此端 →

读路径（查课表、查余量）

每个请求等待时间短，线程池远未占满

最先达到上限的是 CPU 或连接池，在课程实践中计算

写路径（选课提交）

热门课程余量行的行锁使提交串行化：单机约 300 TPS，增加机器后的总上限也仅约 1200 TPS（思考题 1 第五幕，$s \approx 0.25$）

开放瞬间 2000 个提交同时到达，在单机 300 TPS 的上限前排队，排队位置即线程池与 accept 队列

增加线程只是扩大了等待区

要点 表象在线程池，根因在行锁；根本解决方案在数据层（Redis 原子扣减、排队削峰、限流），L-04 讨论第 2 题与思考题 1 第五幕在此汇合

来源：思考题 1 教师要点（第五幕数据）；L-03 Amdahl 定律

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

插曲·延迟的正确表述

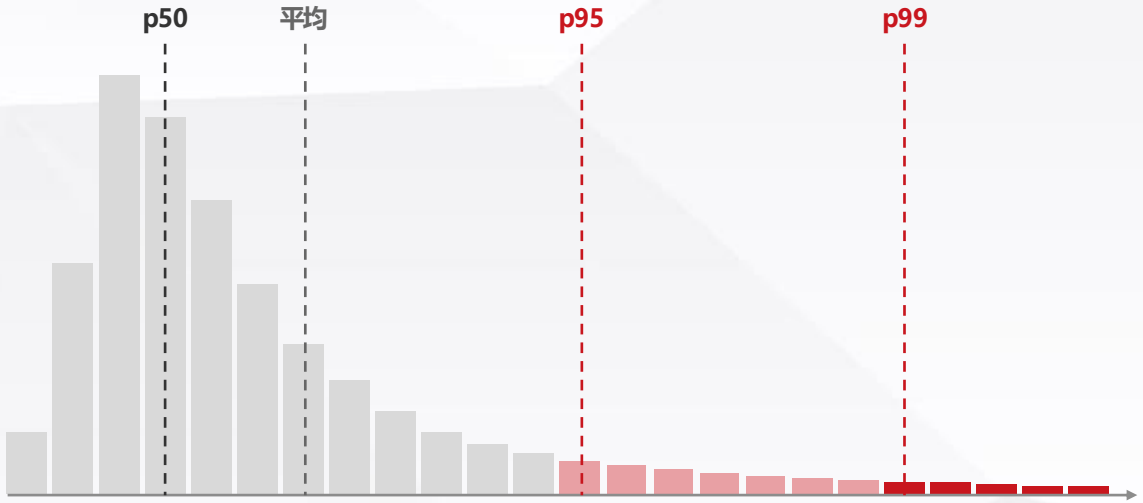
思考题 2 发布

饮水思源·爱国荣校

SJTU

插曲 · 百分位数

平均值的误导：p50 / p95 / p99



大部分请求集中在左侧，长尾向右延伸；平均值被长尾拉高，p50 小于平均值

定义

p99 = 将请求按耗时从小到大排序，位于 99% 位置的耗时，即仅 1% 的请求比它更慢

为什么关注这 1%

一次页面加载涉及数十个请求，一次会话涉及数百个请求，遇到长尾的概率远高于 1%

重度用户遇到长尾的次数最多，而他们往往是最重要的用户

描述延迟应注明百分位；SLO 通常以 p99 而非平均值定义（L-08）

要点 平均值无法刻画延迟分布；p50 反映典型情况，p99 反映长尾

来源：J. Dean & L. Barroso, The Tail at Scale, CACM 56(2), 2013

扇出放大：1 – 0.99^N

一个页面并行调用 N 个服务，每个服务的 p99 为 100 ms

至少遇到一个慢于 100 ms 的调用的概率 = 1 – 0.99^N

N	1 – 0.99^N	含义
1	1%	即该服务的 p99
12	11.4%	单个服务的 p99 约相当于页面的 p89
100	63.4%	大多数页面都会遇到慢调用

前提

各服务的慢事件相互独立

不独立时（如共享同一数据库）概率较低，但慢事件会同时发生

对策仅列出名称，在思考题中讨论

关键路径 · 超时与降级 · 对冲请求 · 重试预算

这是思考题 2 第四题所需的全部数学；其余各题使用 L-03、L-04 的内容即可作答

要点 扇出越大，长尾影响越显著；控制长尾是分布式系统的常规工作

来源：Dean & Barroso (2013)；思考题 2 题面

思考题 2 发布：「14 个服务，160 毫秒」

场景

选课平台三年后拆分为 14 个服务，首页 p50 反而从 180 ms 降至 160 ms

实习生估算为 14 × 20 = 280 ms，错误在哪里?

五幕五题

估算 · RT 由什么决定

开放日 · 缓存与命中率悬崖

提交 · 学生在等待什么

事故 · 尾延迟

评审 · 代价账本

交付要求

小组完成；截止时间 10/15（周四）23:59

题面文件：Q-2.md 与 Q-2.html（可打印版）

题面已给出独立性假设与提示；本讲的两页足以作答

贯穿标准仍是：这项技术解决了什么问题，又增加了什么成本

注 仅介绍场景与交付要求，不讲解题目；10/16 讲评

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、运行时的层级

第六讲 · 软件运行时

饮水思源 • 爱国荣校

SJTU

重新认识软件 – 软件运行时

运行时的层级

裸机 → 虚拟机 → 容器 → 函数

层级	隔离机制	启动时间	密度	解决的问题	增加的成本
裸机	物理隔离	分钟 ~ 天	1	性能无损	利用率低、交付慢
虚拟机	hypervisor，各自独立内核	分钟	十	强隔离、按需创建	每台虚拟机都有独立的内核与系统开销
容器	namespace + cgroup，共享内核	秒	百	打包即制品、秒级起停、密度高	隔离弱于 VM；共享内核扩大了故障面
函数	平台托管的实例	冷启动 100 ms ~ 数秒	按需	可缩容至零，无需管理实例	冷启动；时长 / 内存限制；平台锁定

每下降一层，单个实例更轻量、生命周期更短：L-05 实例轴的时间尺度从月缩短到分钟，再到毫秒
要求也随之提高：启动快、无状态、可随时被终止

判断 这项技术解决了什么问题，又增加了什么成本？

来源：Linux namespaces(7)、cgroups(7)；AWS Lambda 文档（execution environment）；数值为量级示意

容器是受限制的进程

namespace
决定其可见范围
pid / net / mnt / uts / ipc / user

cgroup
决定其可用资源量
cpu / memory / io

镜像
决定其文件系统
即 L-05 所述的不可变制品

不存在特殊的「容器进程」：在宿主机上用 ps 同样可以看到它，只是其可见范围与资源配额受到限制

问题	原因	对策
容器内 CPU 未满，服务却时快时慢	cgroup 的 CPU 配额耗尽后被节流（throttling），在每个周期内暂停执行	检查 throttled 计数；按峰值设定配额，或取消硬性配额
容器被 OOMKilled，宿主机内存却充足	超出的是 cgroup 限制，而非整机内存	按第 7 页核算：堆 + 堆外 + 余量
容器中的 JVM 启动了 64 个 GC 线程	旧版本 JVM 读取的是宿主机的核数	JDK 10+ 自动读取 cgroup 限制；或显式指定

要点 容器化没有改变其进程本质，只是为 CPU、内存、IO 等资源设置了限额；模块 6 资源管理调节的正是这些限额

来源：namespaces(7)、cgroups(7)；Kubernetes docs, Resource Management for Pods and Containers

函数（Serverless）与冷启动

平台代为管理实例

请求到达时才启动实例，无请求时缩容至零；按调用计费

冷启动是 L-05「启动时间是硬约束」的极端形式

从零到可服务需要 100 ms 至数秒，取决于运行时；第二节对比表在此适用：Go / 原生镜像快，JVM 慢

限制与适用场景

单次执行时长、内存上限、并发上限

没有本地持久状态，是 L-05 无状态要求的强制形式

适合：突发、低频、事件驱动的任务（发送短信、生成报表、图片处理）

不适合：长连接、需要预热的热路径

L-02 架构风格比较中 Serverless「节省成本、控制有限」，其中「控制有限」的具体内容即本页所列

要点 函数将实例轴压缩到一次请求的时长；其节省之处与约束条件均源于此

来源：AWS Lambda 文档；L-02 讲义（架构风格比较）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：单机 Tomcat 的运行时画像

饮水思源 · 爱国荣校

第六讲 · 软件运行时

SJTU

参数设定：沿用思考题 1 的单机场景

机器与运行时

4 核 8 GB，一台 Tomcat（JVM），MySQL 部署在另一台（L-04 第一阶段）

Tomcat NIO 连接器默认值：maxThreads 200、maxConnections 8192、acceptCount 100

JVM：-Xmx 4 GB；线程栈默认预留 1 MB

数据库连接池（HikariCP）：默认 10 个连接

单请求耗时与在线人数

一次查询：CPU 3 ms + 等待数据库 8 ms + 其他 1 ms ≈ 占用线程 12 ms

一次提交：CPU 5 ms + 数据库写入 10 ms + 行锁等待（随并发变化）

开放瞬间 2000 人在线，每人 2 ~ 6 条 keep-alive 连接

以上为教学设定值，与思考题 1 压测表一致：800 QPS 正常 / 1200 QPS p95 1.8 s / 1500 QPS 错误率 9%

引例 最先达到上限的是线程池还是行锁？先计算四个上限

四个上限中哪个最先达到?

资源	参数	单请求占用	上限（读路径）
CPU	4 核	3 ms	4 / 0.003 ≈ 1333 QPS
数据库连接池	10 个	8 ms	10 / 0.008 = 1250 QPS
线程池	200	12 ms	200 / 0.012 ≈ 16 700 QPS
客户端连接 / fd	8192 / ulimit	按在线人数计算，而非 QPS	2000 人 × 4 ≈ 8000 条，接近上限
内存	8 GB	≈ 4 GB 堆 + 0.5 GB 堆外	与 QPS 无直接关系；取决于连接数与响应大小

对照思考题 1 压测表：800 QPS 时 CPU 60%、连接池 64%，p95 150 ms，运行正常；1200 QPS 时两者均达 90%～96%，排队延迟非线性激增；1500 QPS 超过上限，错误率 9%，与计算结果一致

要点 读路径最先达到上限的是连接池与 CPU，线程池的上限高出一个数量级；「线程池最先耗尽」在读路径上并不成立

课堂讨论

1 读路径

连接池调至 30 后，下一个达到上限的是哪项？

在不增加机器的前提下，还能如何提升？

2 写路径（选课提交）

为什么线程池反而会先耗尽？

将线程增至 2000 个是否有效？

3 开放瞬间 2000 人 × 4 条连接

先触及的是 maxConnections 还是 ulimit？此时 CPU 处于什么状态？

思考题 1 中的 Nginx 对此有何帮助？

4 更换运行时

将服务改用 Go 实现，或启用虚拟线程：

表中哪些上限发生变化，哪些不变？

安排 每题小组讨论 2 分钟、汇报 1 分钟，其余时间讲评

讨论要点

1 读路径

连接池 30 → 上限 3750 QPS，下一个上限是 CPU 的 1333 QPS

不增加机器时只能降低单请求 CPU 消耗：缓存热数据（L-04 第二阶段）、减少序列化、合并查询

进一步扩展即思考题 1 的 4 台 + Nginx 方案

2 写路径

行锁将总吞吐限制在约 1200 TPS 以内（单机约 300 TPS），2000 个提交在其前排队，排队位置依次为线程池与 accept 队列

2000 个线程只是扩大了等待区，上下文切换与内存开销会先拖垮机器

解决方案在数据层：Redis 原子扣减 / 排队削峰 / 限流

3 连接

ulimit 仍为 1024 → 先出现 Too many open files，CPU 较空闲

提高后接近 maxConnections 8192：新连接在 backlog 中排队，握手超时

Nginx 代替后端持有上万条客户端连接，后端只需处理少量复用的上游连接

4 更换运行时

线程这一层的上限消失或大幅提高

CPU 上限（3 ms/请求）基本不变；连接池上限不变（由数据库端决定）；行锁上限不变

更换运行时改变的是线程成本，不改变 CPU、连接池与锁的上限

结论 线程池是表象，行锁是根因；最先达到上限的资源应通过计算确定，而非凭经验猜测；下一讲系统讨论各类资源的上限

本讲小结

- **运行中的软件是一个进程**
  - 占用三类资源：线程（CPU 时间 + 栈）、内存（堆 + 堆外 + 页缓存）、fd（连接 + 文件）
- **选择运行时即选择资源画像**
  - 并发单位、内存基线、启动时间、多核利用各不相同
- **一请求一线程：λ_max = 线程数 / W**
  - 排队普遍存在，有界队列加超时才能形成背压；连接池按数据库能力确定
- **线程池是表象，行锁是根因**
  - 非阻塞与虚拟线程只改变线程这一层的上限
- **容器是受限制的进程；函数是对启动时间与无状态要求的极端形式**

判断标准 该组件在机器上占用哪些资源？上限由哪种资源决定？

课后思考与参考资料

课后思考

1. 为小组项目的服务绘制运行时画像：使用速查页的命令测量线程数、RSS 与 fd 数，并按「四个上限」表估算哪个上限最先达到。

2. 思考题 2 第四题需用到 1 – 0.99^N：首页同步等待的下游调用有几个？请计算结果。

3. 若将 2025 实验 2 的 Go 后端改为 Java 实现，「四个上限」表中哪些行会发生变化?

参考资料

B. Gregg, Systems Performance: Enterprise and the Cloud, 2nd ed. (2020), Ch. 2、3、6、7

Linux man pages：proc(5)、sched(7)、epoll(7)、namespaces(7)、cgroups(7)、ss(8)

Apache Tomcat 10.1, HTTP Connector（maxThreads / maxConnections / acceptCount）

JEP 444: Virtual Threads；JDK container support (JDK-8146115)

HikariCP wiki, About Pool Sizing；MySQL 8.4 Reference, max_connections

J. Dean & L. Barroso, The Tail at Scale, CACM 56(2), 2013

Kubernetes docs, Resource Management for Pods and Containers；AWS Lambda, Execution environment

The Twelve-Factor App, VIII. Concurrency

感谢聆听

饮 水 思 源　爱 国 荣 校